

Borrowly

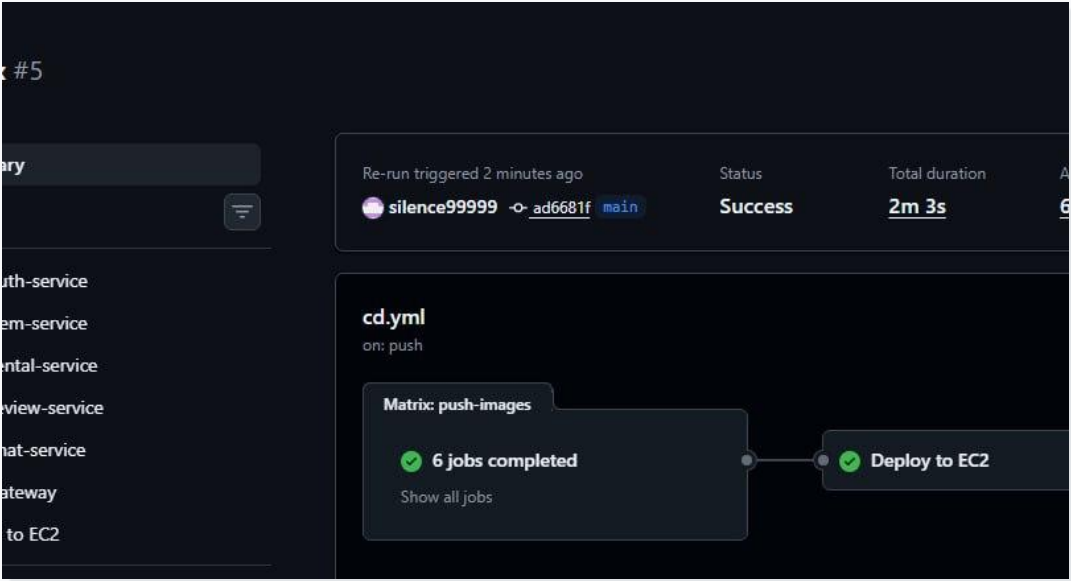
SRE · Microservices · CI/CD

Comprehensive Site Reliability Engineering implementation for a distributed rental marketplace.

Docker · Kubernetes · Terraform · Ansible · GitHub Actions · Prometheus · Grafana

Team
SE-2431 / SE-2430

Project
Peer-to-peer rental marketplace



✓	CD / Deploy to EC2 (push)	Successful in 31s	Details
✓	CI / Docker build auth-service (push)	Successful in 49s	Details
✓	CI / Docker build chat-service (push)	Successful in 58s	Details
✓	CI / Docker build gateway (push)	Successful in 23s	Details
✓	CI / Docker build item-service (push)	Successful in 45s	Details
✓	CI / Docker build rental-service (push)	Successful in 50s	Details

Executive snapshot

What the final system demonstrates

6 services

Auth, item, rental, review, chat, notification

19 green checks

Tests, builds, image pushes and EC2 deploy

2m 03s deploy

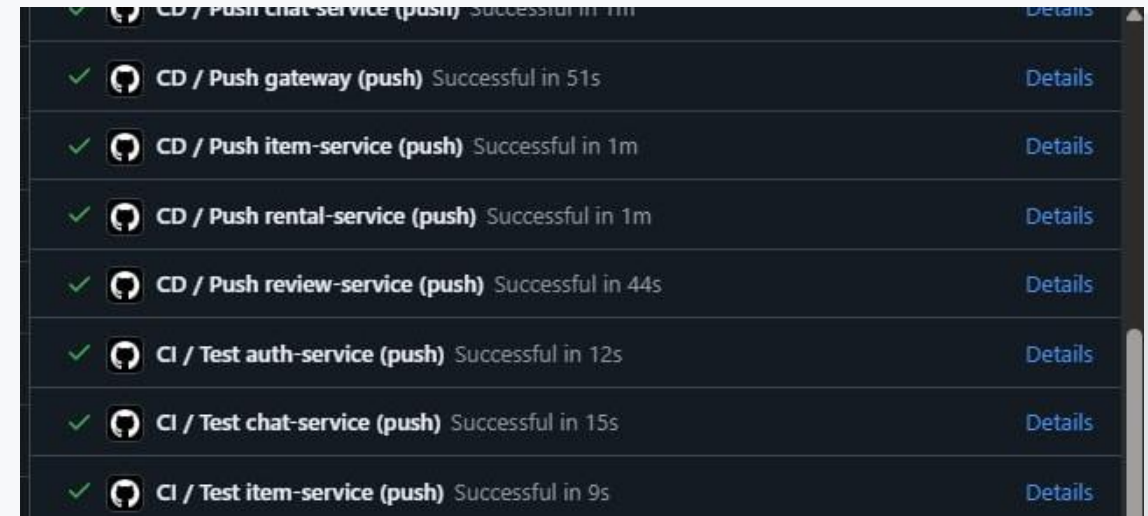
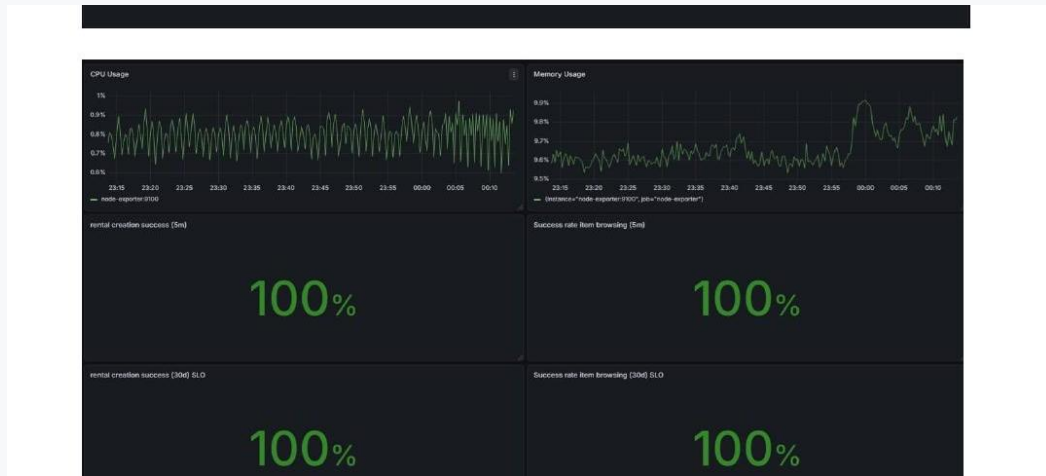
End-to-end CD workflow run

12 alert rules

Availability, latency, error budget and host metrics

6 min recovery

Simulated production incident resolved



Live system and pipeline evidence: monitoring dashboard + all checks passing.

Project goal

Turn a microservices app into a reliable production platform

Reliability by design

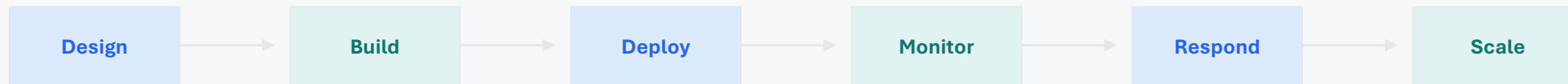
Each service exposes health and metrics endpoints, with database isolation and restart policies.

Repeatable operations

Infrastructure, server setup and deployment are automated instead of manual.

Fast feedback loop

GitHub Actions validates each change before deployment to EC2.



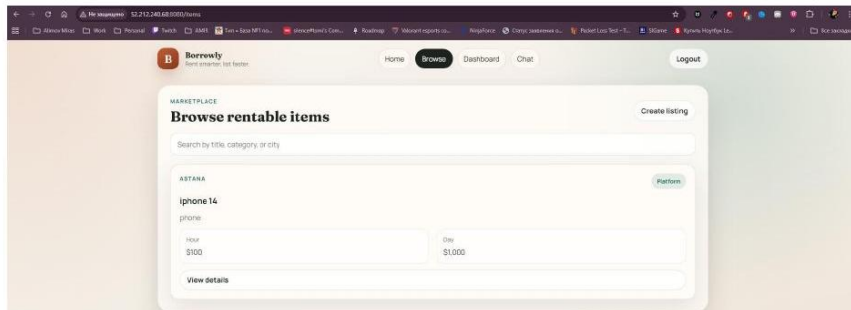
The presentation follows the same SRE lifecycle used in the implementation.

Borrowly product context

Peer-to-peer marketplace as the reliability target

```
- db-auth  
- db-items  
- db-rentals  
- db-reviews  
- db-chat  
restart: unless-stopped
```

3.3 All 6 Services Running



List items

Users publish rentable items with images and pickup points.

Rent & pay

Rental service manages orders, pricing and statuses.

Chat

User-to-user messaging supports coordination.

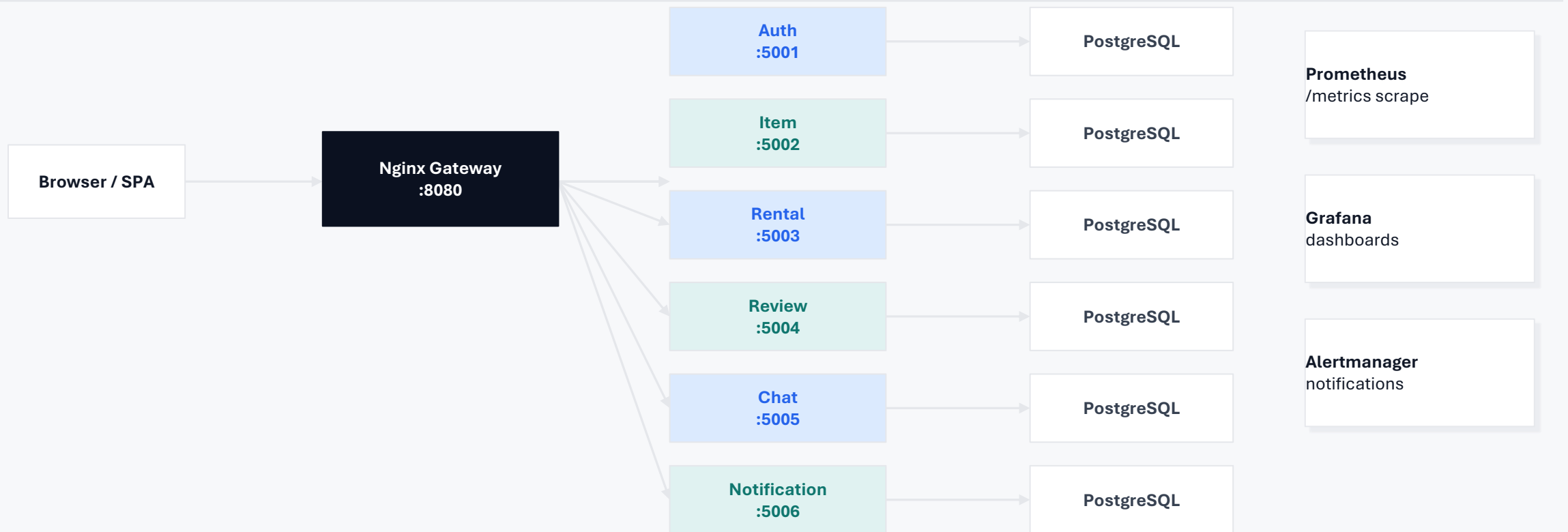
Review

Ratings and reviews close the marketplace loop.

Reliability matters because every feature depends on several services working together through the gateway and databases.

Architecture

Gateway-first microservices with isolated persistence



The design avoids shared databases between services, making failures easier to isolate and recover.

Microservices map

Six independent responsibilities, one unified product

5001

Auth

JWT, registration, email verification

5002

Item

CRUD, image upload, pickup points

5003

Rental

Orders, pricing, reminder worker

5004

Review

Item reviews and star ratings

5005

Chat

User-to-user in-app messaging

5006

Notification

Rental confirmations and reminders

Every service is containerized and paired with its own PostgreSQL database, then exposed through the gateway.

Infrastructure & orchestration

The same application can run locally, clustered, and in cloud

Docker Compose

Local development and full-stack testing

Docker Swarm

Cluster-ready deployment model

Kubernetes + HPA

Production-grade scaling path

Terraform

AWS EC2, security group and outputs

Ansible

Server setup, roles and automated deployment

```
+ kms_key_id      = (known after apply)
+ tags_all        = (known after apply)
+ throughput      = (known after apply)
+ volume_id       = (known after apply)
+ volume_size     = 20
+ volume_type     = "gp3"
}

Plan: 1 to add, 0 to change, 0 to destroy.

Changes to Outputs:
+ app_url           = (known after apply)
+ grafana_url       = (known after apply)
+ instance_public_dns = (known after apply)
+ instance_public_ip = (known after apply)
+ prometheus_url    = (known after apply)
+ ssh_command       = (known after apply)

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.
```

```
item-service-8579b4968b-z428d 1/1 Running 1 (7m41s ago) 7m35s
notification-service-9d7f49998-8zqck 1/1 Running 0 7m27s
notification-service-9d7f49998-nqpqg 1/1 Running 1 (7m41s ago) 7m53s
prometheus-79c8b7578d-rbf2b 1/1 Running 0 7m52s
rental-service-65d4b646d5-hbvkq 1/1 Running 1 (7m41s ago) 7m52s
review-service-5f84d7d8cf-2nhj2 1/1 Running 1 (7m41s ago) 7m53s
review-service-5f84d7d8cf-9z1vl 1/1 Running 0 7m27s
```

NAME	REFERENCE	TARGETS	RINPODS	MAXPODS	REPLICAS	AGE
auth-service-hpa	Deployment/auth-service	cpu: <unknown>/70%	1	4	2	25s
chat-service-hpa	Deployment/chat-service	cpu: <unknown>/70%	1	4	2	25s
gateway-hpa	Deployment/gateway	cpu: <unknown>/70%	1	4	2	25s
item-service-hpa	Deployment/item-service	cpu: <unknown>/70%	1	4	2	25s
notification-service-hpa	Deployment/notification-service	cpu: <unknown>/70%	1	4	2	25s
rental-service-hpa	Deployment/rental-service	cpu: <unknown>/70%	1	4	2	25s
review-service-hpa	Deployment/review-service	cpu: <unknown>/70%	1	4	2	25s

10.4 Comparison

Feature	Docker Compose	Docker Swarm	Kubernetes
Setup complexity	Low	Medium	High
Auto-scaling	Manual	Manual (service scale)	Automatic (HPA)

One-command mindset

Provision, configure and deploy with reproducible scripts.

Secure exposure

Public app gateway; admin tools restricted to admin CIDR.

Continuous Integration

Parallel checks make every push safe

```
jobs:
  test-go:
    name: Test ${matrix.service}
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false
      matrix:
        service:
          - auth-service
          - item-service
          - rental-service
          - review-service
          - chat-service
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-go@v5
        with:
          go-version: '1.24'
          cache-dependency-path: services/${matrix.service}/go.sum
      - name: vet
        working-directory: services/${matrix.service}
        run: go vet ./...
      - name: test
        working-directory: services/${matrix.service}
        run: go test ./...

  build-frontend:
    name: Build frontend
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: npm
```

test-go

Matrix over 5 Go services: go vet + go test

build-frontend

Node 20, npm ci, TypeScript check, production build

docker-build

BuildKit builds 6 images with GitHub Actions cache, push disabled

```
60     runs-on: ubuntu-latest
61     needs: [test-go, build-frontend]
62     strategy:
63       fail-fast: false
64       matrix:
```


Continuous Deployment

Images are pushed, then EC2 is updated over SSH

```
4   push:
5     branches: [main]
6   workflow_dispatch:
7
8   permissions:
9     contents: read
10    packages: write
11
12  jobs:
13    push-images:
14      name: Push ${matrix.service}
15      runs-on: ubuntu-latest
16      strategy:
17        fail-fast: false
18        matrix:
19          include:
20            - service: auth-service
21              context: services/auth-service
22            - service: item-service
23              context: services/item-service
24            - service: rental-service
25              context: services/rental-service
26            - service: review-service
27              context: services/review-service
28            - service: chat-service
29              context: services/chat-service
30            - service: gateway
31              context: .
32            dockerfile: gateway/Dockerfile
33      steps:
34        - uses: actions/checkout@v4
35
36        - uses: docker/setup-buildx-action@v4
37
38        - uses: docker/login-action@v4
39          with:
40            registry: ghcr.io
41            username: ${github.actor}
42            password: ${secrets.GHCR_PUSH_TOKEN}
43
```

```
44  - id: meta
45    uses: docker/metadata-action@v0
46    with:
47      images: ghcr.io/${github.repository_owner}/rent-items-${matrix.service}
48      tags: |
49        type=sha,prefix=
50        type=raw,value=latest
51
52  - uses: docker/build-push-action@v0
53    with:
54      context: ${matrix.context}
55      file: ${matrix.dockerfile} || Format('{0}/Dockerfile', matrix.context)
56      push: true
57      tags: ${steps.meta.outputs.tags}
58      labels: ${steps.meta.outputs.labels}
59      cache-from: type=gha
60      cache-to: type=gha,mode=max
61
62  deploy:
63    name: Deploy to EC2
64    runs-on: ubuntu-latest
65    needs: push-images
66    environment: production
67    steps:
68      - uses: actions/checkout@v4
69
70      - name: Deploy via SSH
71        uses: appleboy/ssh-action@v1.0.3
72        with:
73          host: ${secrets.EC2_HOST}
74          username: ${secrets.EC2_USER}
75          key: ${secrets.EC2_SSH_PRIVATE_KEY}

```

GHCR registry

Images tagged by Git SHA and latest.

Deploy job

SSH to EC2, pull images, restart stack.

Secrets safe

Tokens and keys stored only in GitHub Secrets.

Pipeline evidence

Final run: all checks passed

✓  CD / Deploy to EC2 (push) Successful in 31s	Details
✓  CI / Docker build auth-service (push) Successful in 49s	Details
✓  CI / Docker build chat-service (push) Successful in 58s	Details
✓  CI / Docker build gateway (push) Successful in 23s	Details
✓  CI / Docker build item-service (push) Successful in 45s	Details
✓  CI / Docker build rental-service (push) Successful in 50s	Details

✓  CD / Push item-service (push) Successful in 1m	Details
✓  CD / Push rental-service (push) Successful in 1m	Details
✓  CD / Push review-service (push) Successful in 44s	Details
✓  CI / Test auth-service (push) Successful in 12s	Details
✓  CI / Test chat-service (push) Successful in 15s	Details

#5

Re-run triggered 2 minutes ago

 silence99999  ad6681f [main](#)

Status: **Success** Total duration: **2m 3s**

cd.yml
on: push

Matrix: push-images

✓ 6 jobs completed
[Show all jobs](#)

✓ Deploy to EC2

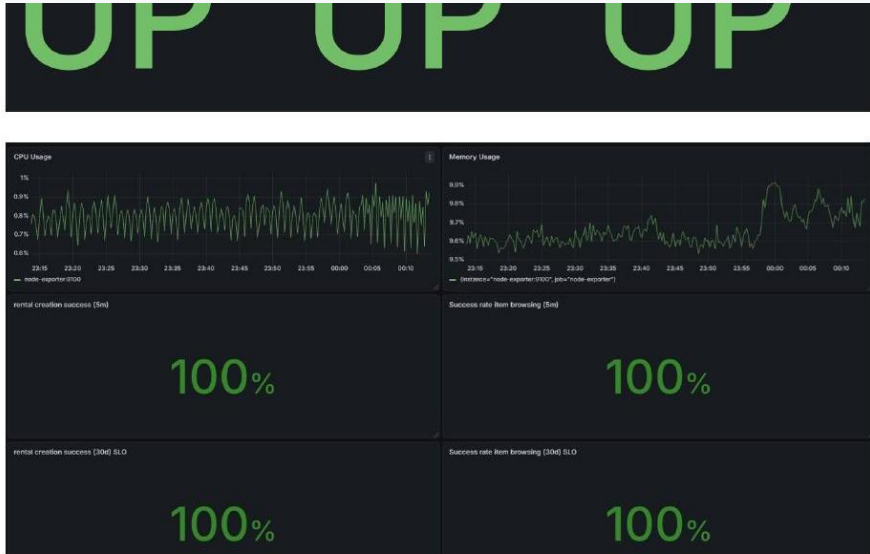
19
successful checks

2m 03s
total duration

31s
deploy to EC2

Observability stack

Metrics, dashboards and alerts close the feedback loop



Prometheus

Scrapes all services and node exporter every 15 seconds.

Grafana

Service health, request rate, latency and resource dashboards.

Alertmanager

Routes incidents to the responsible engineer.

Node Exporter

Host CPU, memory and disk metrics.

Monitoring is not a separate add-on: metrics and health checks are part of every service contract.

SLIs & SLOs

Reliability targets are explicit and measurable

Objective	Target	Alert threshold
Availability	$\geq 99\%$	up == 0 for 1 minute
P95 latency	$\leq 200\text{ ms}$	Rental P95 > 1s for 5 minutes
Error rate	$\leq 1\%$	5xx rate > 5% for 2 minutes
Rental success	$\geq 90\%$	failure rate > 10% for 2 minutes

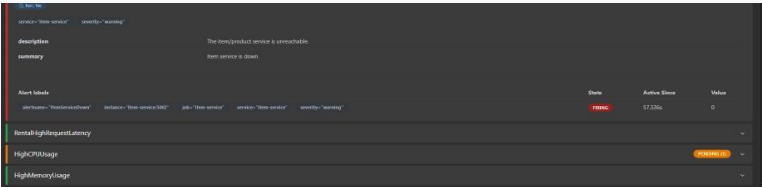
Availability
Prometheus target reachability

Latency
95th-percentile response time

Error budget
5xx errors per second

Incident response

A controlled failure tested detection and recovery



7.4 Root Cause Analysis

Docker logs revealed the exact failure.

```
ubuntu@ip-172-31-30-230:~/advanced_final$ docker logs advanced_final-item-service-1
2026/04/29 15:15:31 item-service: db ping failed:dial tcp 172.18.0.4:54: connect: connection refused
2026/04/29 15:15:32 item-service: db ping failed:dial tcp 172.18.0.4:54: connect: connection refused
2026/04/29 15:15:32 item-service: db ping failed:dial tcp 172.18.0.4:54: connect: connection refused
2026/04/29 15:15:33 item-service: db ping failed:dial tcp 172.18.0.4:54: connect: connection refused
2026/04/29 15:15:34 item-service: db ping failed:dial tcp 172.18.0.4:54: connect: connection refused
2026/04/29 15:15:36 item-service: db ping failed:dial tcp 172.18.0.4:54: connect: connection refused
2026/04/29 15:15:39 item-service: db ping failed:dial tcp 172.18.0.4:54: connect: connection refused
2026/04/29 15:15:46 item-service: db ping failed:dial tcp 172.18.0.4:54: connect: connection refused
2026/04/29 15:15:59 item-service: db ping failed:dial tcp 172.18.0.4:54: connect: connection refused
2026/04/29 15:16:25 item-service: db ping failed:dial tcp 172.18.0.4:54: connect: connection refused
```

7.5 Recovery

The DSN was corrected and the database container restarted.

```
ubuntu@ip-172-31-30-230:~/advanced_final$ docker compose up --build db-items
[+] Running 1/0
  ✓ Container advanced_final-db-items-1 Created
Attaching to db-items-1
db-items-1 |
db-items-1 | PostgreSQL Database directory appears to contain a database; Skipping initialization
db-items-1 |
db-items-1 | 2026-04-29 15:22:20.339 UTC [1] LOG: starting PostgreSQL 16.15 on x86_64-pc-linux-musl, compiled by gcc (Alpine 15.2.0) 16.2.0, 64-bit
db-items-1 | 2026-04-29 15:22:20.359 UTC [1] LOG: listening on IPv4 address "0.0.0.0", port 5432
db-items-1 | 2026-04-29 15:22:20.359 UTC [1] LOG: listening on IPv6 address ":::", port 5432
db-items-1 | 2026-04-29 15:22:20.366 UTC [1] LOG: listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
db-items-1 | 2026-04-29 15:22:20.394 UTC [28] LOG: database system was shut down at 2026-04-29 15:19:03 UTC
db-items-1 | 2026-04-29 15:22:20.364 UTC [1] LOG: database system is ready to accept connections
```

```
$ docker exec final-item-service-1 kill 1
$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                                     NAMES
f66859e0b0a   final-gateway                        /docker-entrypoint.sh   51 minutes ago Up 51 minutes   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp   final-gateway-1
f6a6d3f0a3e   final-rental-service                /app/service            51 minutes ago Up 51 minutes (healthy: starting) 5053/tcp   final-rental-service-1
76dcd320d23   grafana/grafana:latest              /run.sh                 51 minutes ago Up 51 minutes                                     final-grafana-1
8734a4d4a4e   prom/prometheus:latest              /entrypoint.sh          51 minutes ago Up 51 minutes                                     final-prometheus-1
b09ffac199b   final-chat-service                  /app/service            51 minutes ago Up 51 minutes (healthy) 5053/tcp   final-chat-service-1
247d7020139   final-review-service                /app/service            51 minutes ago Up 51 minutes (healthy) 5053/tcp   final-review-service-1
5d579d45922   final-item-service                  /app/service            51 minutes ago Up 51 minutes (healthy) 5053/tcp   final-item-service-1
```

T+0 DB DSN misconfigured

T+1 Prometheus fires alert

T+2 Alertmanager notifies engineer

T+3 Logs reveal DB ping failure

T+5 Configuration corrected

T+6 Service recovers

Root cause
Manual .env typo

Outcome
Recovered in 6 minutes



Remediation & prevention

The incident became automation

```
$ ./scripts/validate-config.sh .env
=== Validating configuration: .env ===

-- Database credentials --
[ OK ] AUTH_DB_USER
[ OK ] AUTH_DB_PASSWORD
[ OK ] AUTH_DB_NAME
[ OK ] AUTH_DB_DSN
[ OK ] ITEM_DB_USER
[ OK ] ITEM_DB_PASSWORD
[ OK ] ITEM_DB_NAME
[ OK ] ITEM_DB_DSN
[ OK ] RENTAL_DB_USER
[ OK ] RENTAL_DB_PASSWORD
[ OK ] RENTAL_DB_NAME
[ OK ] RENTAL_DB_DSN
[ OK ] REVIEW_DB_USER
[ OK ] REVIEW_DB_PASSWORD
[ OK ] REVIEW_DB_NAME
[ OK ] REVIEW_DB_DSN
[ OK ] CHAT_DB_USER
[ OK ] CHAT_DB_PASSWORD
[ OK ] CHAT_DB_NAME
[ OK ] CHAT_DB_DSN

-- Service configuration --
[ OK ] AUTH_SERVICE_URL
[ OK ] ITEM_SERVICE_URL
[ OK ] SECRET

-- SMTP configuration --
[ OK ] SMTP_HOST
[ OK ] SMTP_PORT
[ OK ] SMTP_USERNAME
[ OK ] SMTP_PASSWORD
[ OK ] SMTP_SENDER

-- pgAdmin --
[ OK ] POSTGRES_USER
[ OK ] POSTGRES_PASSWORD

=== VALIDATION PASSED. All required variables are set. ===
```

8. Assignment 5 — Infrastructure as Code (Terraform)

8.1 Terraform File Structure

17/34

Config validation

scripts/validate-config.sh checks all required variables and DSNs before deployment.

Health dependencies

Services wait for healthy databases before starting.

Per-service alerts

Down alerts detect outages within one minute.

Deployment discipline

CI/CD reduces manual production changes.

Load testing & capacity planning

Tests revealed bottlenecks before production scaling

9.3 Grafana During Load Test



Load script

Concurrent HTTP requests to all API endpoints.

Bottleneck

rental-service identified as the scaling priority.

Scaling path

Horizontal replicas through Swarm/Kubernetes HPA.

Capacity plan

Monitoring guides CPU, memory and replica decisions.

The goal was not just passing tests, but proving the system can be measured and scaled responsibly.

Repository deliverables

A complete production-style project, not just application code

[services/](#)

Go microservices

[frontend/](#)

React + TypeScript SPA

[gateway/](#)

Nginx reverse proxy

[observability/](#)

Prometheus + Grafana + alerts

[terraform/](#)

AWS infrastructure

[ansible/](#)

Server setup and deployment

[.github/workflows/](#)

CI/CD automation

[docker-compose.yaml](#)

Local full-stack orchestration

GitHub repository: [silence99999/borrowly](#)

Source code, workflows, infrastructure files and technical evidence are versioned together.

Final results

Requirements covered across the SRE lifecycle

Architecture

6 independent services + 6 isolated databases

Automation

Terraform provisioning + Ansible setup

Delivery

19 checks pass on every push to main

Observability

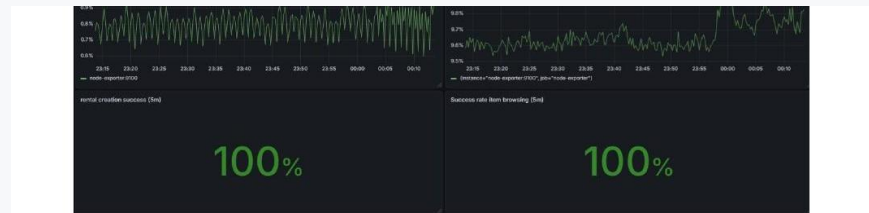
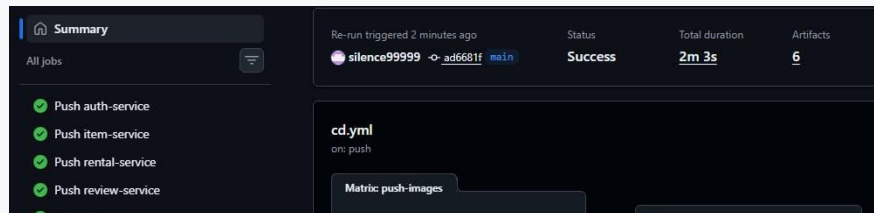
Prometheus, Grafana, Alertmanager, 12 rules

Resilience

Incident detected <1 min and recovered in 6 min

Scaling

Load-tested and capacity-planned



Thank you

Borrowly proves the full path from code to reliable operation: build, deploy, monitor, respond, improve.

Reliable
health checks, SLOs, alerts

Repeatable
IaC + configuration
management

Fast
2-minute CI/CD feedback loop

All checks have passed			
19 successful checks			
✓	 CI / Build frontend (push)	Successful in 14s	Details
✓	 CD / Deploy to EC2 (push)	Successful in 31s	Details
✓	 CI / Docker build auth-service (push)	Successful in 49s	Details
✓	 CI / Docker build chat-service (push)	Successful in 58s	Details
✓	 CI / Docker build gateway (push)	Successful in 23s	Details
✓	 CI / Docker build item-service (push)	Successful in 45s	Details
✓	 CI / Docker build rental-service (push)	Successful in 50s	Details
✓	 CI / Docker build review-service (push)	Successful in 47s	Details
✓	 CD / Push auth-service (push)	Successful in 1m	Details
✓	 CD / Push chat-service (push)	Successful in 1m	Details

